

services\data\data_processor.py

```
import os
import json
import gzip
from typing import List, Dict, Any
from datetime import datetime
import config

class DataProcessor:
    """Process and extract insights from historical NVD data - LOCAL FILES ONLY"""

    def __init__(self):
        self.historical_dir = config.NVD_HISTORICAL_DIR
        self.processed_dir = config.NVD_PROCESSED_DIR
        # CRITICAL: In-memory cache for SINGLE year only
        self._year_cache = {}
        self._max_cache_size = 1 # Only cache 1 year at a time

    print(f"[Historical] DataProcessor initialized")
    print(f"[Historical] Local directory: {self.historical_dir}")

    # Check what files are available
    self._check_available_files()

    def _check_available_files(self):
        """Check what historical files are available"""
        if not os.path.exists(self.historical_dir):
            print(f"[Historical] Directory does not exist: {self.historical_dir}")
            print(f"[Historical] Creating directory...")
            os.makedirs(self.historical_dir, exist_ok=True)
            return

        files = os.listdir(self.historical_dir)
        if files:
            print(f"[Historical] Found {len(files)} files: {files}")
        else:
            print(f"[Historical] WARNING: No files found in {self.historical_dir}")
            print(f"[Historical] Please add CVE-XXXX.json or CVE-XXXX.json.gz files to this directory")

    def get_year_data(self, year: int) -> List[Dict]:
        """Load CVEs from a specific year file - WITH CACHING"""
        # Check cache first
        year_str = str(year)
        if year_str in self._year_cache:
            print(f"[Historical] Using cached data for {year}")
            return self._year_cache[year_str]

        # Load from local file
        data = self._load_year_file(year)

        # Cache management - only keep 1 year in memory
        if len(self._year_cache) >= self._max_cache_size:
            # Clear oldest cache entry
            oldest_year = next(iter(self._year_cache))
            print(f"[Historical] Evicting {oldest_year} from cache to save memory")
            del self._year_cache[oldest_year]

        # Store in cache
        self._year_cache[year_str] = data
        print(f"[Historical] Cached {len(data)} CVEs for {year}")

        return data

    def _load_year_file(self, year: int) -> List[Dict]:
        """Load CVEs from a specific year file - LOCAL FILES ONLY

        This method now supports both compressed (.json.gz) and uncompressed (.json) files.
        """
        try:
            # Try different file patterns - INCLUDING GZIPPED FILES
            possible_filenames = [
                f"CVE-{year}.json.gz", # Try gzipped first (most common)
            ]
        
```

```

f"CVE-{year}.json",
f"nvdcve-1.1-{year}.json.gz",
f"nvdcve-1.1-{year}.json",
f"nvdcve-2.0-{year}.json.gz",
f"nvdcve-2.0-{year}.json",
]

# Check if directory exists
if not os.path.exists(self.historical_dir):
    print(f"[Historical] Directory does not exist: {self.historical_dir}")
    return []

# Find the first available file
target_file = None
filename_used = None
is_gzipped = False

for filename in possible_filenames:
    file_path = os.path.join(self.historical_dir, filename)
    if os.path.exists(file_path):
        target_file = file_path
        filename_used = filename
        is_gzipped = filename.endswith('.gz')
        print(f"[Historical] Found file: {filename} (gzipped={is_gzipped})")
        break

if not target_file:
    print(f"[Historical] No file found for {year}. Tried: {possible_filenames}")
    print(f"[Historical] Available files: {os.listdir(self.historical_dir)}")
    return []

print(f"[Historical] Loading file: {target_file}")

# Check file size
file_size = os.path.getsize(target_file)
print(f"[Historical] File size: {file_size:,} bytes ({file_size/1024/1024:.2f} MB)")

# Load the file - HANDLE BOTH GZIPPED AND UNCOMPRESSED
if is_gzipped:
    print(f"[Historical] Opening gzipped file...")
    with gzip.open(target_file, 'rt', encoding='utf-8') as f:
        data = json.load(f)
    print(f"[Historical] Successfully loaded gzipped JSON file for {year}")
else:
    print(f"[Historical] Opening uncompressed file...")
    with open(target_file, 'r', encoding='utf-8') as f:
        data = json.load(f)
    print(f"[Historical] Successfully loaded JSON file for {year}")

# DEBUG: Print top-level structure
print(f"[Historical] DEBUG: JSON top-level keys: {list(data.keys())}")
print(f"[Historical] DEBUG: JSON type: {type(data)}")

# Process CVEs based on detected format
cves = []

# Try JSON 2.0 format first (newer format)
vulnerabilities = data.get("vulnerabilities", [])
print(f"[Historical] DEBUG: vulnerabilities field exists: {vulnerabilities is not None}")
print(f"[Historical] DEBUG: vulnerabilities length: {len(vulnerabilities) if vulnerabilities else 0}")

if vulnerabilities:
    print(f"[Historical] Found {len(vulnerabilities)} vulnerabilities in JSON 2.0 format")

# DEBUG: Show first item structure
if len(vulnerabilities) > 0:
    first_item = vulnerabilities[0]
    print(f"[Historical] DEBUG: First vulnerability keys: {list(first_item.keys()) if isinstance(first_item, dict) else 'NOT A DICT'}")
    if isinstance(first_item, dict) and 'cve' in first_item:
        print(f"[Historical] DEBUG: First CVE ID: {first_item.get('cve', {}).get('id', 'NO ID')}")

processed_count = 0
failed_count = 0

```

```

for item in vulnerabilities:
    processed = self._process_json_2_0_cve(item)
    if processed:
        cves.append(processed)
        processed_count += 1
    else:
        failed_count += 1

print(f"[Historical] DEBUG: Processed {processed_count} CVEs, {failed_count} failed")
else:
    # Try JSON 1.1 format (legacy format)
    cve_items = data.get("CVE_Items", [])
    print(f"[Historical] DEBUG: CVE_Items field exists: {cve_items is not None}")
    print(f"[Historical] DEBUG: CVE_Items length: {len(cve_items) if cve_items else 0}")

    if cve_items:
        print(f"[Historical] Found {len(cve_items)} CVE_Items in JSON 1.1 format")

    # DEBUG: Show first item structure
    if len(cve_items) > 0:
        first_item = cve_items[0]
        print(f"[Historical] DEBUG: First CVE_Item keys: {list(first_item.keys()) if isinstance(first_item, dict) else 'NOT A DICT'}")

    processed_count = 0
    failed_count = 0
    for item in cve_items:
        processed = self._process_json_1_1_cve(item)
        if processed:
            cves.append(processed)
            processed_count += 1
        else:
            failed_count += 1

    print(f"[Historical] DEBUG: Processed {processed_count} CVEs, {failed_count} failed")
    else:
        print(f"[Historical] ERROR: No recognized data format found")
        print(f"[Historical] Top-level keys: {list(data.keys())}")

    # Try to give more helpful debugging info
    for key in list(data.keys())[:5]: # First 5 keys
        value = data[key]
        print(f"[Historical] DEBUG: Key '{key}' = {type(value)} (length: {len(value) if hasattr(value, '__len__') else 'N/A'})")

    print(f"[Historical] Successfully processed {len(cves)} CVEs for {year}")

    if len(cves) == 0:
        print(f"[Historical] WARNING: Zero CVEs were processed! Check JSON structure.")

    return cves

except Exception as e:
    print(f"[Historical] ERROR loading {year}: {e}")
    import traceback
    traceback.print_exc()
    return []

def _process_json_2_0_cve(self, item: Dict) -> Dict:
    """Process JSON 2.0 format CVE - MEMORY OPTIMIZED"""
    try:
        cve_data = item.get("cve", {})
        if not cve_data:
            print(f"[Historical] DEBUG: Item missing 'cve' field, keys: {list(item.keys())}")
            return None

        cve_id = cve_data.get("id", "")

        if not cve_id:
            print(f"[Historical] DEBUG: CVE data missing 'id', keys: {list(cve_data.keys())}")
            return None

        # Extract English description
        description = ""
        for desc in cve_data.get("descriptions", []):
            if desc.get("lang") == "en":
                description = desc.get("value", "")

```

```

break

# Extract severity
severity = "UNKNOWN"
metrics = cve_data.get("metrics", {})

for version in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2"]:
    if version in metrics and metrics[version]:
        try:
            metric = metrics[version][0]
            cvss_data = metric.get("cvssData", {})
            potential_severity = cvss_data.get("baseSeverity", "")

            if potential_severity and potential_severity.upper() != "UNKNOWN":
                severity = potential_severity.upper()
                break
        except (IndexError, KeyError):
            continue

# Extract CWE
cwe = None
for weakness in cve_data.get("weaknesses", []):
    for desc in weakness.get("description", []):
        if desc.get("lang") == "en":
            value = desc.get("value", "")
            if value.startswith("CWE"):
                cwe = value
                break
    if cwe:
        break

return {
    "ID": cve_id,
    "Description": description,
    "Severity": severity,
    "CWE": cwe,
    "Published": cve_data.get("published", ""),
    "lastModified": cve_data.get("lastModified", ""),
    "References": [],
    "Products": [],
    "CVSS_Score": None,
    "metrics": {}
}

except Exception as e:
    print(f"[Historical] ERROR processing JSON 2.0 CVE: {e}")
    return None

def _process_json_1_1_cve(self, item: Dict) -> Dict:
    """Process JSON 1.1 format CVE - MEMORY OPTIMIZED"""
    try:
        cve_data = item.get("cve", {})
        if not cve_data:
            print(f"[Historical] DEBUG: Item missing 'cve' field")
            return None

        cve_id = cve_data.get("CVE_data_meta", {}).get("ID", "")

        if not cve_id:
            return None

        # Extract English description
        description = ""
        descriptions = cve_data.get("description", {}).get("description_data", [])
        for desc in descriptions:
            if desc.get("lang") == "en":
                description = desc.get("value", "")
                break

        # Extract severity
        severity = "UNKNOWN"
        impact = item.get("impact", {})

        if "baseMetricV3" in impact:
            severity = impact["baseMetricV3"].get("cvssV3", {}).get("baseSeverity",
                "UNKNOWN").upper()
        elif "baseMetricV2" in impact:

```

```

score = impact["baseMetricV2"].get("cvssV2", {}).get("baseScore", 0)
if score >= 7.0:
    severity = "HIGH"
elif score >= 4.0:
    severity = "MEDIUM"
else:
    severity = "LOW"

# Extract CWE
cwe = None
problem_types = cve_data.get("problemtype", {}).get("problemtype_data", [])
for problem_type in problem_types:
    for desc in problem_type.get("description", []):
        if desc.get("lang") == "en":
            value = desc.get("value", "")
            if value.startswith("CWE"):
                cwe = value
                break
    if cwe:
        break

return {
    "ID": cve_id,
    "Description": description,
    "Severity": severity,
    "CWE": cwe,
    "Published": cve_data.get("publishedDate", ""),
    "lastModified": cve_data.get("lastModifiedDate", ""),
    "References": [],
    "Products": [],
    "CVSS_Score": None,
    "metrics": {}
}

except Exception as e:
    print(f"[Historical] ERROR processing JSON 1.1 CVE: {e}")
    return None

# Global historical loader instance
historical_loader = DataProcessor()

```